ntic master
revolucionamos la comunicación

UNIVERSIDAD COMPLUTENSE DE MADRID



# Clustering


Luis Garmendia Salvador

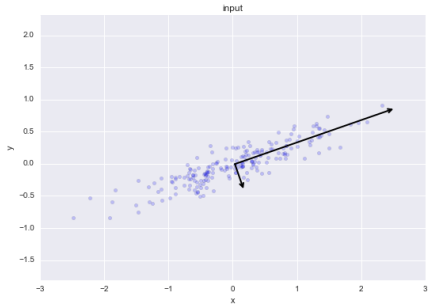
Departamento de Ingeniería del Software e Inteligencia Artificial
Facultad de Informática, UCM

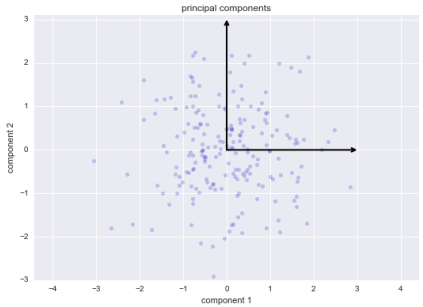
# 3

# Clustering

Algoritmos de clustering y algunas aplicaciones:

- PCA
  - Reducción de la Dimensión
  - Transformación Afin (cambio de base)
  - Clasificación de dígitos
  - Filtrado de ruido
- K-Means
  - Algoritmo
  - Ejemplo de clasificación
  - Problemas (centroides, parámetro, fronteras lineales)
  - Clasificación de dígitos
  - Aplicación para la compresión de imágenes




# Principal Component Analysis

Luís Garmendia

# Principal Component Analysis

PCA es un algoritmo de la reducción de la dimension, con aplicaciones a la visualización, filtrado de ruido, feature engineering….

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set()
```

# Principal Component Analysis

200 puntos:

```
rng = np.random.RandomState(1)
X = np.dot(rng.rand(2, 2), rng.randn(2, 200)).T
plt.scatter(X[:, 0], X[:, 1])
plt.axis('equal');
```



# Principal Component Analysis

Claramente X e Y están relacionados

PCA intenta aprender la lrelación entre X e Y



# Principal Component Analysis

La realación se encuentra en los ejes principals de los datos para describer el dataset y en la varianza explicable.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
pca.fit(X)
```

# Principal Component Analysis

print(pca.components_)

print(pca.explained_variance_)

```
In [417]: from sklearn.decomposition import PCA
          pca = PCA(n_components=2)
          pca.fit(X)

Out[417]: PCA(copy=True, iterated_power='auto', n_components=2, random_state=None,
              svd_solver='auto', tol=0.0, whiten=False)

In [418]: print(pca.components_)

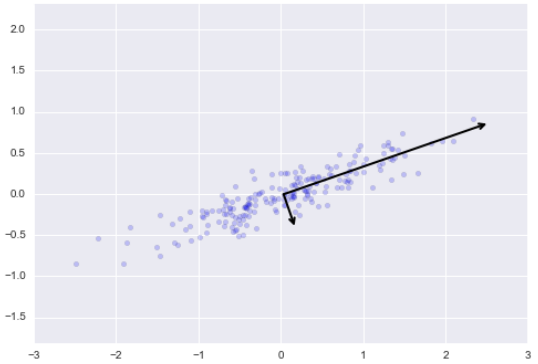
          [[-0.94446029 -0.32862557]
           [-0.32862557  0.94446029]]

In [419]: print(pca.explained_variance_)

          [0.7625315 0.0184779]
```

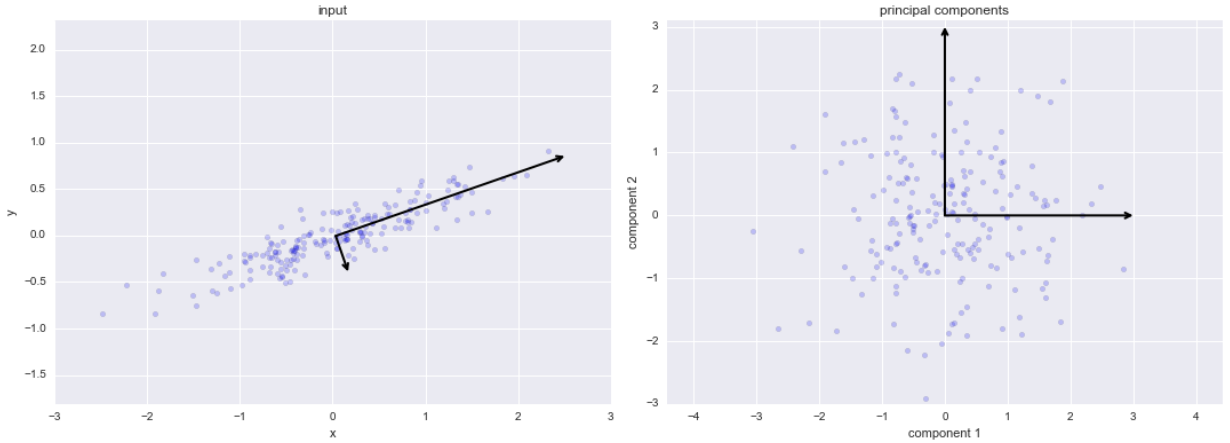
# Principal Component Analysis visualización

```
def draw_vector(v0, v1, ax=None):
    ax = ax or plt.gca()
    arrowprops=dict(arrowstyle='->',
                    linewidth=2,
                    shrinkA=0, shrinkB=0)
    ax.annotate('', v1, v0, arrowprops=arrowprops)
# plot data
plt.scatter(X[:, 0], X[:, 1], alpha=0.2)
for length, vector in zip(pca.explained_variance_, pca.co
    v = vector * 3 * np.sqrt(length)
    draw_vector(pca.mean_, pca.mean_ + v)
plt.axis('equal');
```



# Principal Component Analysis
# Transformación Afín (cambio de base)

La transformación de los datos con base a los ejes principals es una transformación afin.



# Principal Component Analysis
# PCA como redución de la dimensión

Los componentes elegidos **maximizan la varianza**

```
pca = PCA(n_components=1)
pca.fit(X)
X_pca = pca.transform(X)
print("original shape: ", X.shape)
print("transformed shape:", X_pca.shape)
```

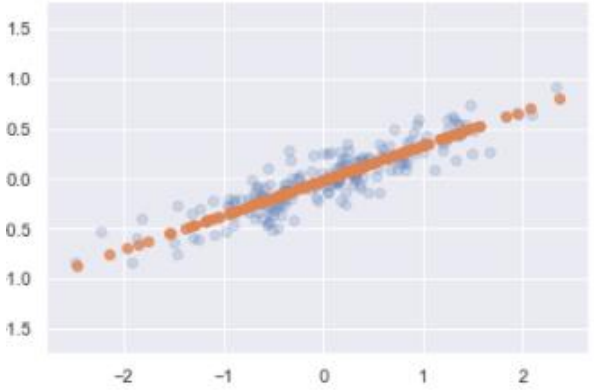
```
pca = PCA(n_components=1)
pca.fit(X)
X_pca = pca.transform(X)
print("original shape: ", X.shape)
print("transformed shape:", X_pca.shape)
```

```
original shape:  (200, 2)
transformed shape: (200, 1)
```

# Principal Component Analysis
# PCA como redución de la dimensión

Reducción a una sóla dimensión

```
X_new = pca.inverse_transform(X_pca)
plt.scatter(X[:, 0], X[:, 1], alpha=0.2)
plt.scatter(X_new[:, 0], X_new[:, 1], alpha=0.8)
plt.axis('equal');
```



# PCA y los dígitos

```
from sklearn.datasets import load_digits
digits = load_digits()
digits.data.shape
```

```
In [426]: from sklearn.datasets import load_digits
          digits = load_digits()
          digits.data.shape

Out[426]: (1797, 64)
```

# PCA y los dígitos

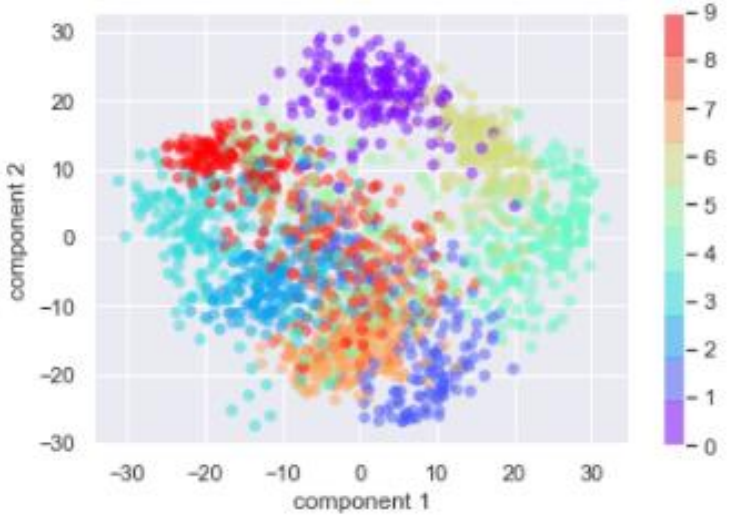
```
pca = PCA(2) # project from 64 to 2 dimensions
projected = pca.fit_transform(digits.data)
print(digits.data.shape)
print(projected.shape)
```

```
pca = PCA(2) # project from 64 to 2 dimensions
projected = pca.fit_transform(digits.data)
print(digits.data.shape)
print(projected.shape)
```

```
(1797, 64)
(1797, 2)
```

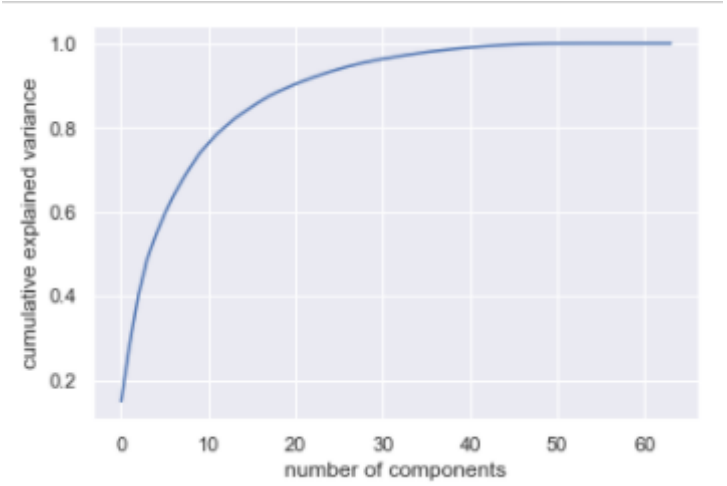
# PCA y los dígitos

```
plt.scatter(projected[:, 0], projected[:, 1], c=digits.target,
edgecolor='none', alpha=0.5, cmap=plt.cm.get_cmap('rainbow', 10))
plt.xlabel('component 1')
plt.ylabel('component 2')
plt.colorbar();
```



# Elección del número de componentes

```
pca = PCA().fit(digits.data)
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel('number of components')
plt.ylabel('cumulative explained variance');
```



# PCA y filtrado de ruido

Se añade ruido al azar….

```
np.random.seed(42)
noisy = np.random.normal(digits.data, 4)
plot_digits(noisy)
```



# PCA y filtrado de ruido

Proyección que preserve el 50% de la varianza:

```
pca = PCA(0.50).fit(noisy)
pca.n_components_
```

```
In [440]: pca = PCA(0.50).fit(noisy)
          pca.n_components_

Out[440]: 12
```

# PCA y filtrado de ruido

El 50%  de la varianza está en 12 components

```
components = pca.transform(noisy)
filtered = pca.inverse_transform(components)
plot_digits(filtered)
```

```
In [441]: components = pca.transform(noisy)
          filtered = pca.inverse_transform(components)
          plot_digits(filtered)
```



# Eigenfaces
# PCA as feature selector

```
from sklearn.datasets import fetch_lfw_people
faces = fetch_lfw_people(min_faces_per_person=60)
print(faces.target_names)
print(faces.images.shape)
```

```
In [443]: from sklearn.datasets import fetch_lfw_people
          faces = fetch_lfw_people(min_faces_per_person=60)
          print(faces.target_names)
          print(faces.images.shape)

          ['Ariel Sharon' 'Colin Powell' 'Donald Rumsfeld' 'George W Bush'
           'Gerhard Schroeder' 'Hugo Chavez' 'Junichiro Koizumi' 'Tony Blair']
          (1348, 62, 47)
```

# Eigenfaces
# PCA as feature selector

```
fig, ax = plt.subplots(3, 5)
for i, axi in enumerate(ax.flat):
    axi.imshow(faces.images[i], cmap='bone')
    axi.set(xticks=[], yticks=[], xlabel=faces.target_names[faces.target[i]])
plt.show()
```

```
In [444]: fig, ax = plt.subplots(3, 5)
          for i, axi in enumerate(ax.flat):
              axi.imshow(faces.images[i], cmap='bone')
              axi.set(xticks=[], yticks=[], xlabel=faces.target_names[faces.target[i]])
```



# Eigenfaces
# PCA as feature selector

[62 × 47] 3,000 pixels.
Sejarlo en 150 componentes y clasificar con SVC: pipeline

```
from sklearn.svm import SVC
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
pca = PCA(n_components=150, whiten=True,
random_state=42)
svc = SVC(kernel='rbf', class_weight='balanced')
model = make_pipeline(pca, svc)
```

# PCA as feature selector train_test_split

```
from sklearn.model_selection import
train_test_split

Xtrain, Xtest, ytrain, ytest = (faces.data, faces.target,
random_state=42)
```

# PCA as feature selector
# Determinar los mejores parámetros

```
from sklearn.model_selection import GridSearchCV
param_grid = {'svc__C': [1, 5, 10, 50],
              'svc__gamma': [0.0001, 0.0005, 0.001, 0.005]}
grid = GridSearchCV(model, param_grid)
%time grid.fit(Xtrain, ytrain)
print(grid.best_params_)
```

```
from sklearn.model_selection import GridSearchCV
param_grid = {'svc__C': [1, 5, 10, 50],
              'svc__gamma': [0.0001, 0.0005, 0.001, 0.005]}
grid = GridSearchCV(model, param_grid)

%time grid.fit(Xtrain, ytrain)
print(grid.best_params_)
```

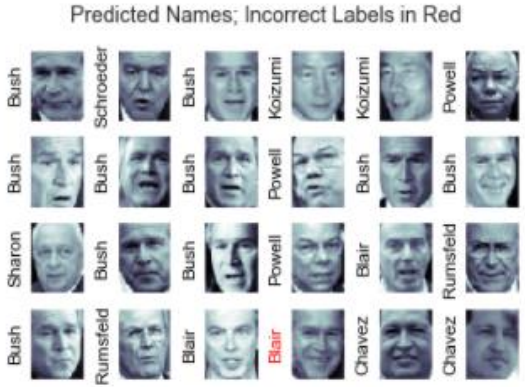
```
Wall time: 34.7 s
{'svc__C': 10, 'svc__gamma': 0.001}
```

# PCA as feature selector test data

```
model = grid.best_estimator_
yfit = model.predict(Xtest)
```

# PCA as feature selector
# Predicción de caras

```
fig, ax = plt.subplots(4, 6)
for i, axi in enumerate(ax.flat):
    axi.imshow(Xtest[i].reshape(62, 47), cmap='bone')
    axi.set(xticks=[], yticks=[])
    axi.set_ylabel(faces.target_names[yfit[i]].split()[-1],
                   color='black' if yfit[i] == ytest[i] else 'red')
fig.suptitle('Predicted Names; Incorrect Labels in Red', size=14);
```



# PCA as feature selector performance

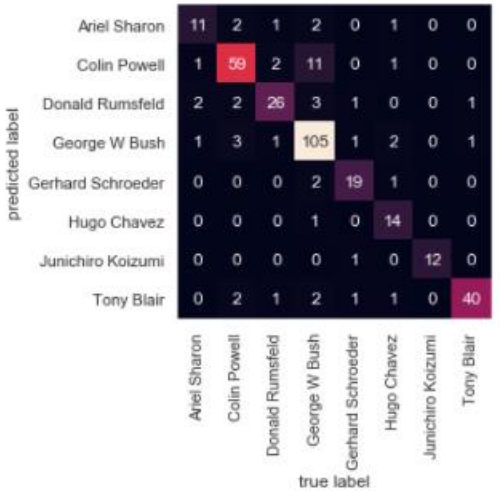
```
from sklearn.metrics import classification_report
print(classification_report(ytest, yfit, target_names=faces.target_names))
```

```
from sklearn.metrics import classification_report
print(classification_report(ytest, yfit, target_names=faces.target_names))
```

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Ariel Sharon | 0.65 | 0.73 | 0.69 | 15 |
| Colin Powell | 0.80 | 0.87 | 0.83 | 68 |
| Donald Rumsfeld | 0.74 | 0.84 | 0.79 | 31 |
| George W Bush | 0.92 | 0.83 | 0.88 | 126 |
| Gerhard Schroeder | 0.86 | 0.83 | 0.84 | 23 |
| Hugo Chavez | 0.93 | 0.70 | 0.80 | 20 |
| Junichiro Koizumi | 0.92 | 1.00 | 0.96 | 12 |
| Tony Blair | 0.85 | 0.95 | 0.90 | 42 |
| accuracy | | | 0.85 | 337 |
| macro avg | 0.83 | 0.84 | 0.84 | 337 |
| weighted avg | 0.86 | 0.85 | 0.85 | 337 |

# PCA as feature selector confusion matrix

```
from sklearn.metrics import confusion_matrix
mat = confusion_matrix(ytest, yfit)
sns.heatmap(mat.T, square=True, annot=True, fmt='d',
cbar=False,
        xticklabels=faces.target_names,
        yticklabels=faces.target_names)
plt.xlabel('true label')
plt.ylabel('predicted label');
```



# Eigenfaces
# Dibujando las 150 componentes

from sklearn.decomposition import PCA

pca = PCA(150)

pca.fit(faces.data)

```
In [460]: from sklearn.decomposition import PCA
          pca = PCA(150)
          pca.fit(faces.data)

Out[460]: PCA(copy=True, iterated_power='auto', n_components=150, random_state=None,
              svd_solver='auto', tol=0.0, whiten=False)
```

# Eigenfaces

```
fig, axes = plt.subplots(3, 8, figsize=(9, 4),
                         subplot_kw={'xticks':[], 'yticks':[]},
                         gridspec_kw=dict(hspace=0.1, wspace=0.1))
for i, ax in enumerate(axes.flat):
  ax.imshow(pca.components_[i].reshape(62, 47), cmap='bone')
```

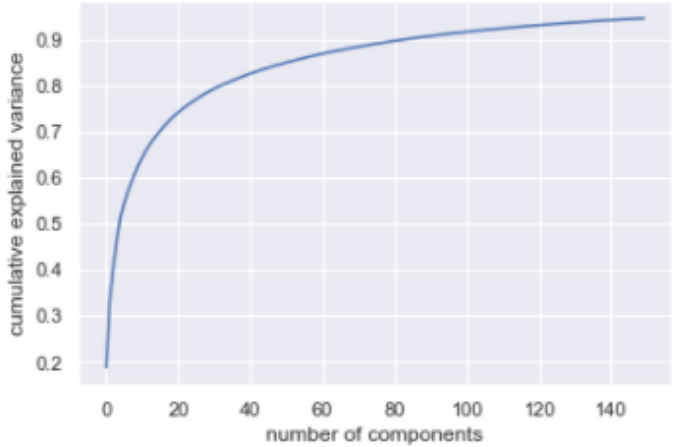
```
In [461]: fig, axes = plt.subplots(3, 8, figsize=(9, 4),
                                   subplot_kw={'xticks':[], 'yticks':[]},
                                   gridspec_kw=dict(hspace=0.1, wspace=0.1))
          for i, ax in enumerate(axes.flat):
              ax.imshow(pca.components_[i].reshape(62, 47), cmap='bone')
```



# Eigenfaces
# Variabilidad preservada

```
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel('number of components')
plt.ylabel('cumulative explained variance');
```

```
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel('number of components')
plt.ylabel('cumulative explained variance');
```



# Eigenfaces
# Imagenes desde las 150 componentes

```
pca = PCA(150).fit(faces.data)
components = pca.transform(faces.data)
projected = pca.inverse_transform(components)
```

```
# Plot the results
fig, ax = plt.subplots(2, 10, figsize=(10, 2.5),
                       subplot_kw={'xticks':[], 'yticks':[]},
                       gridspec_kw=dict(hspace=0.1, wspace=0.1))
for i in range(10):
    ax[0, i].imshow(faces.data[i].reshape(62, 47), cmap='binary_r')
    ax[1, i].imshow(projected[i].reshape(62, 47), cmap='binary_r')

ax[0, 0].set_ylabel('full-dim\ninput')
ax[1, 0].set_ylabel('150-dim\nreconstruction');
```



# Eigenfaces
# Imagenes desde las 150 componentes

```
# Compute the components and projected faces
pca = PCA(150).fit(faces.data)
components = pca.transform(faces.data)
projected = pca.inverse_transform(components)
```

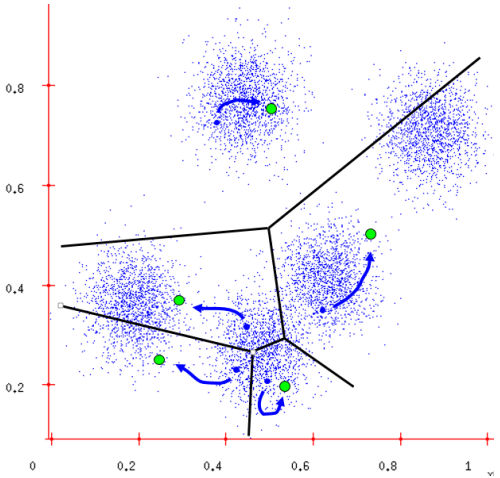
```
# Plot the results
fig, ax = plt.subplots(2, 10, figsize=(10, 2.5),
                       subplot_kw={'xticks':[], 'yticks':[]},
                       gridspec_kw=dict(hspace=0.1, wspace=0.1))
for i in range(10):
    ax[0, i].imshow(faces.data[i].reshape(62, 47), cmap='binary_r')
    ax[1, i].imshow(projected[i].reshape(62, 47), cmap='binary_r')

ax[0, 0].set_ylabel('full-dim\ninput')
ax[1, 0].set_ylabel('150-dim\nreconstruction');
```

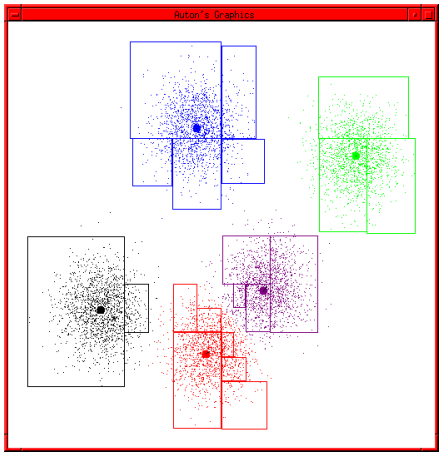


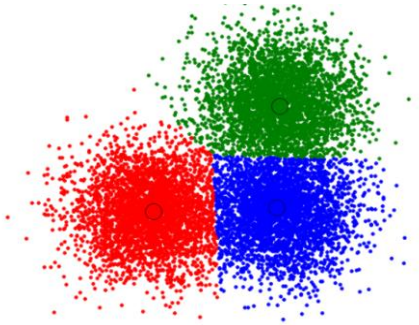
# Eigenfaces
# Imagenes desde las 150 componentes





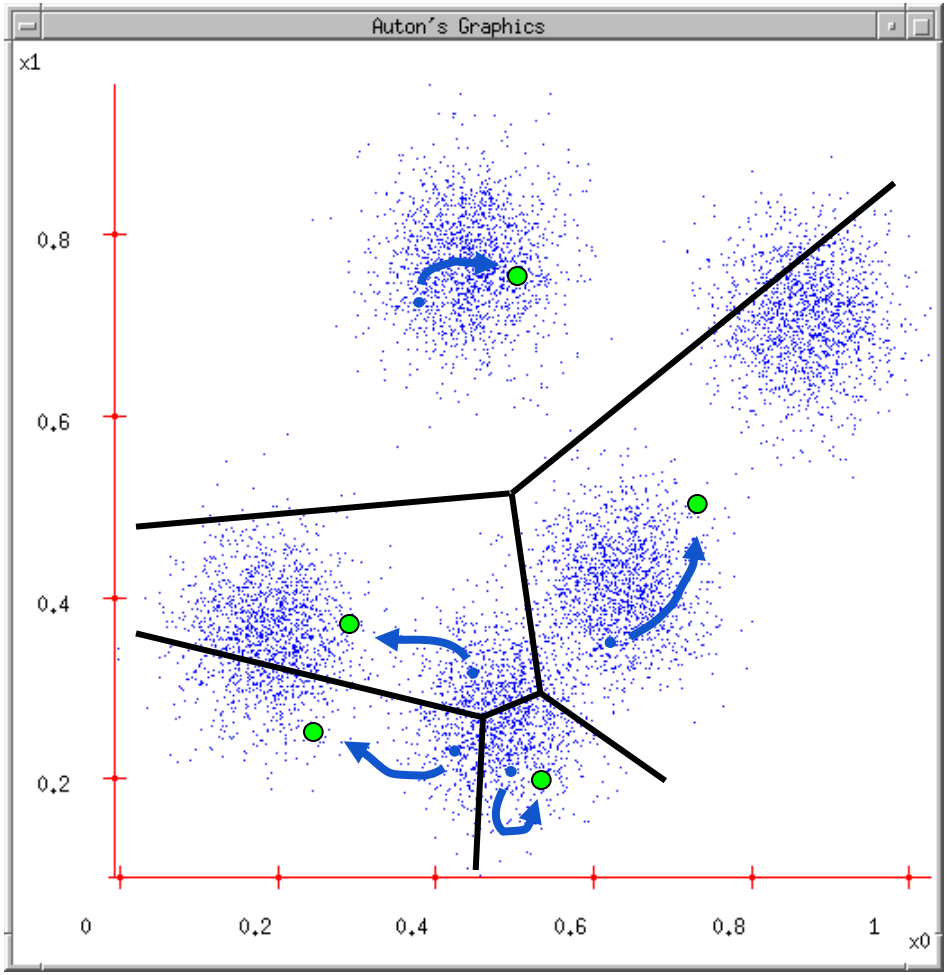

# K-means clustering



## K-means

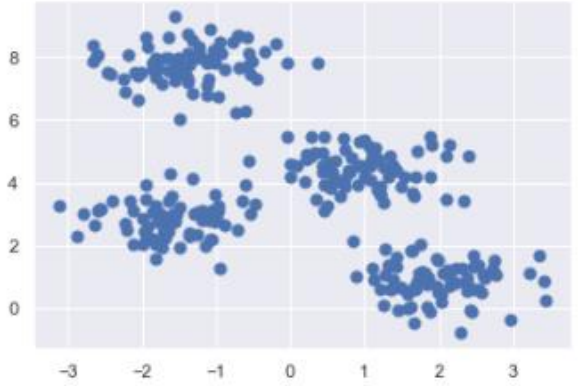
1. Preguntar número de clusters *(e.g. k=5)*
2. Escoger al azar k cluster Center locations
3. Cada punto escoge el centro más cercano
4. Cada centro escoge el centroide de sus puntos
5. Repetir….



# K-means clustering

```
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns; sns.set() # for plot styling
import numpy as np

from sklearn.datasets.samples import make_blobs
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.60,
random_state=0)
plt.scatter(X[:, 0], X[:, 1], s=50);
```

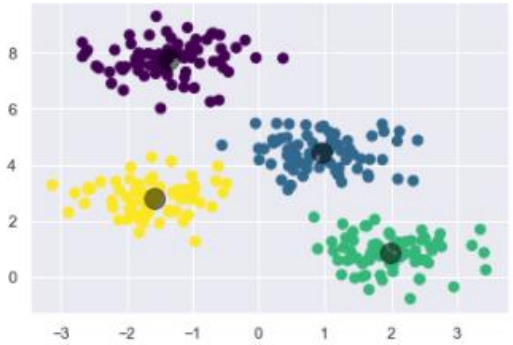


# K-means clustering

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=4)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)
```

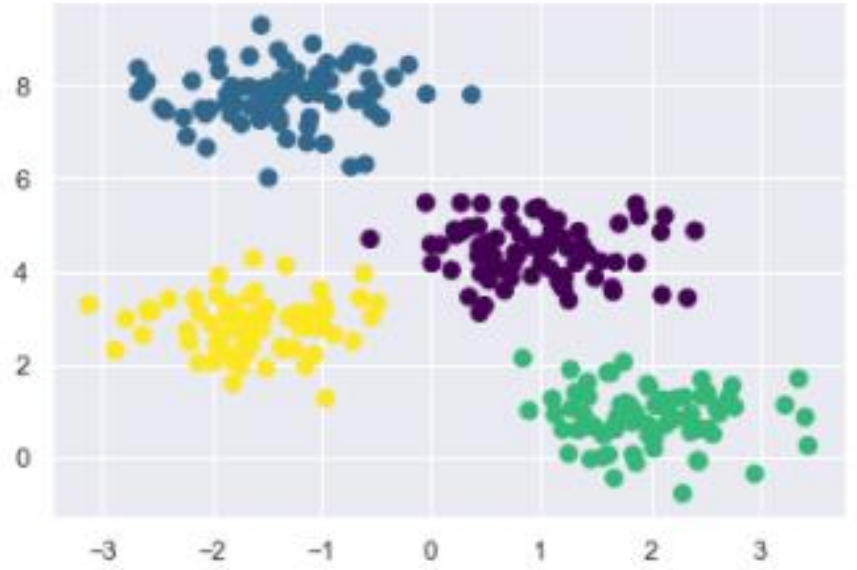
Visualización de centroides:

```
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='black', s=200, alpha=0.5);
```



# K-Means implementado

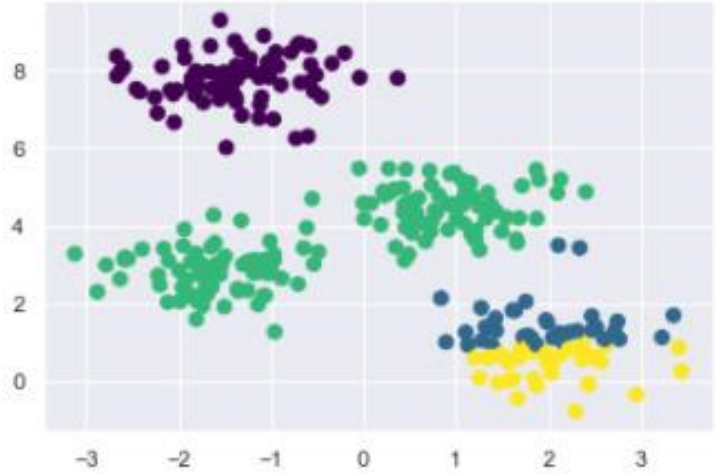
```
def find_clusters(X, n_clusters, rseed=2):
    # 1. Randomly choose clusters
    rng = np.random.RandomState(rseed)
    i = rng.permutation(X.shape[0])[:n_clusters]
    centers = X[i]
    while True:
        # 2a. Assign labels based on closest center
        labels = pairwise_distances_argmin(X, centers)
        # 2b. Find new centers from means of points
        new_centers = np.array([X[labels == i].mean(0)
                                for i in range(n_clusters)])
        # 2c. Check for convergence
        if np.all(centers == new_centers):
            break
        centers = new_centers
    return centers, labels
centers, labels = find_clusters(X, 4)
plt.scatter(X[:, 0], X[:, 1], c=labels,
            s=50, cmap='viridis');
```



# Problema de centroides random

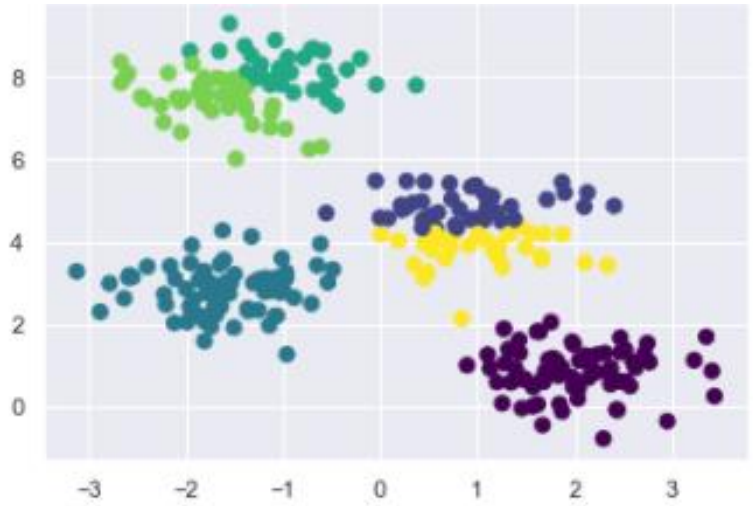
No se asegura la mejor solución, se puede converger a soluciones no óptimas.

```
centers, labels = find_clusters(X, 4, rseed=0)
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis');
```



# Se debe indicar el número de clusteres

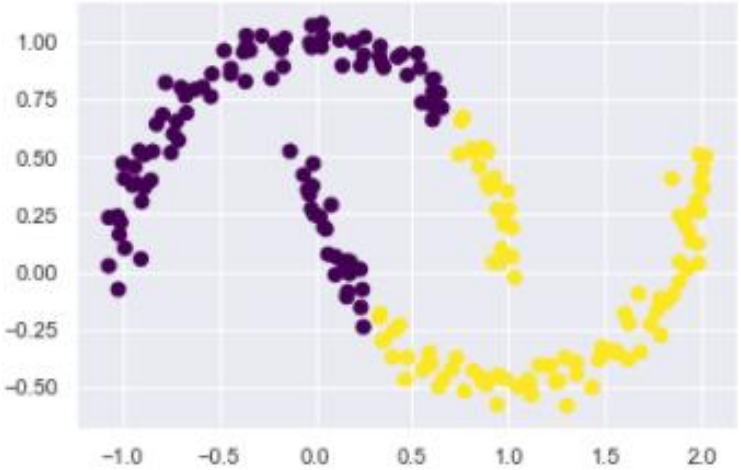
```
labels = KMeans(6, random_state=0).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis');
```



# Problema de las fronteras lineales

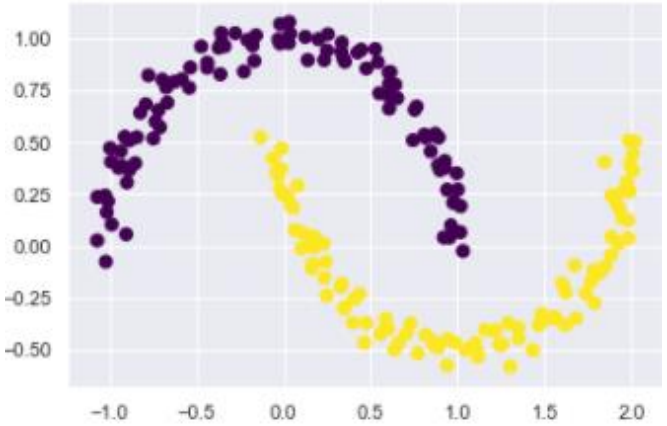
```
from sklearn.datasets import make_moons
X, y = make_moons(200, noise=.05, random_state=0)

labels = KMeans(2, random_state=0).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis');
```



# Trasnformación de kernel: SpectralClustering

```
from sklearn.cluster import SpectralClustering
model = SpectralClustering(n_clusters=2,
affinity='nearest_neighbors', assign_labels='kmeans')
labels = model.fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis');
```



# k-means y dígitos

```
from sklearn.datasets import load_digits
digits = load_digits()
digits.data.shape
```

```
(1797, 64)
```

# k-means y dígitos

Se clasifican los 1797 dígitos en 10 clases

kmeans = KMeans(n_clusters=10, random_state=0)
clusters = kmeans.fit_predict(digits.data)
kmeans.cluster_centers_.shape

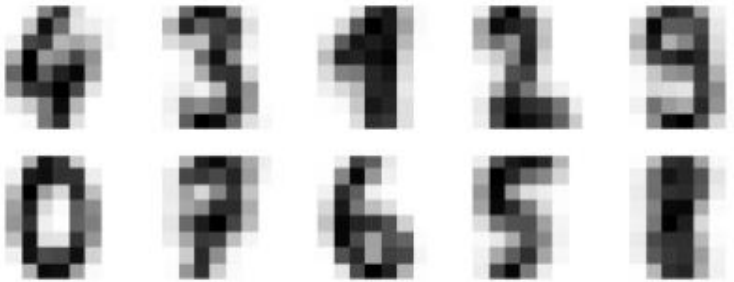
```
kmeans = KMeans(n_clusters=10, random_state=0)
clusters = kmeans.fit_predict(digits.data)
kmeans.cluster_centers_.shape
```

```
(10, 64)
```

# k-means y dígitos

Se dibujan los centroides:

```
 fig, ax = plt.subplots(2, 5, figsize=(8, 3))
centers = kmeans.cluster_centers_.reshape(10, 8, 8)
for axi, center in zip(ax.flat, centers):
                axi.set(xticks=[], yticks=[])
                axi.imshow(center, interpolation='nearest',
cmap=plt.cm.binary)
```



# k-means y digitos accuracy

Transformamos las etiquetas de los clusteres

```
from scipy.stats import mode
labels = np.zeros_like(clusters)
for i in range(10):
        mask = (clusters == i)
        labels[mask] = mode(digits.target[mask])[0]
```

```
clusters.shape
```

```
(1797,)
```

```
np.zeros_like(clusters).shape
```

```
(1797,)
```

```
np.zeros_like(clusters)
```

```
array([0, 0, 0, ..., 0, 0, 0])
```

```
from scipy.stats import mode
labels = np.zeros_like(clusters)
for i in range(10):
    mask = (clusters == i)
    labels[mask] = mode(digits.target[mask])[0]
```

```
labels
```

```
array([0, 8, 8, ..., 8, 9, 9])
```

```
mask
```

```
array([False,  True,  True, ...,  True, False, False])
```

```
labels.shape
```

```
(1797,)
```

# k-means on digits
# accuracy 80%

Se estudia la precisión

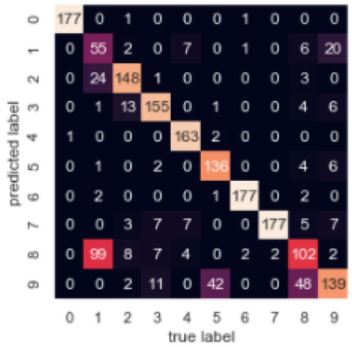
```
from sklearn.metrics import accuracy_score
accuracy_score(digits.target, labels)
```

```
In [501]: from sklearn.metrics import accuracy_score
          accuracy_score(digits.target, labels)

Out[501]: 0.7952142459654981
```

# k-means on digits
# Matriz de confusión

```
from sklearn.metrics import confusion_matrix
mat = confusion_matrix(digits.target, labels)
sns.heatmap(mat.T, square=True, annot=True, fmt='d', cbar=False,
xticklabels=digits.target_names, yticklabels=digits.target_names)
plt.xlabel('true label')
plt.ylabel('predicted label');
```



# *k*-means para la compresión de imágenes

```
# Note: this requires the ``pillow`` package to be installed
from sklearn.datasets import load_sample_image
china = load_sample_image("china.jpg")
ax = plt.axes(xticks=[], yticks=[])
ax.imshow(china);
```



# *k*-means para la compresión de imágenes

La imagen es un array tridimensional (height, width, RGB),
RGB: red/blue/green con valores de 0 a 255:



```
china.shape
```

```
(427, 640, 3)
```

# *k*-means para la compresión de imágenes

Reshape a (427 * 640, 3)  Colores entre 0 y 1.

```
data = china / 255.0 # use 0...1 scale
data = data.reshape(427 * 640, 3)
data.shape
```

```
In [505]: data = china / 255.0 # use 0...1 scale
          data = data.reshape(427 * 640, 3)
          data.shape

Out[505]: (273280, 3)

In [506]: data

Out[506]: array([[0.68235294, 0.78823529, 0.90588235],
                 [0.68235294, 0.78823529, 0.90588235],
                 [0.68235294, 0.78823529, 0.90588235],
                 ...,
                 [0.16862745, 0.19215686, 0.15294118],
                 [0.05098039, 0.08235294, 0.02352941],
                 [0.05882353, 0.09411765, 0.02745098]])
```

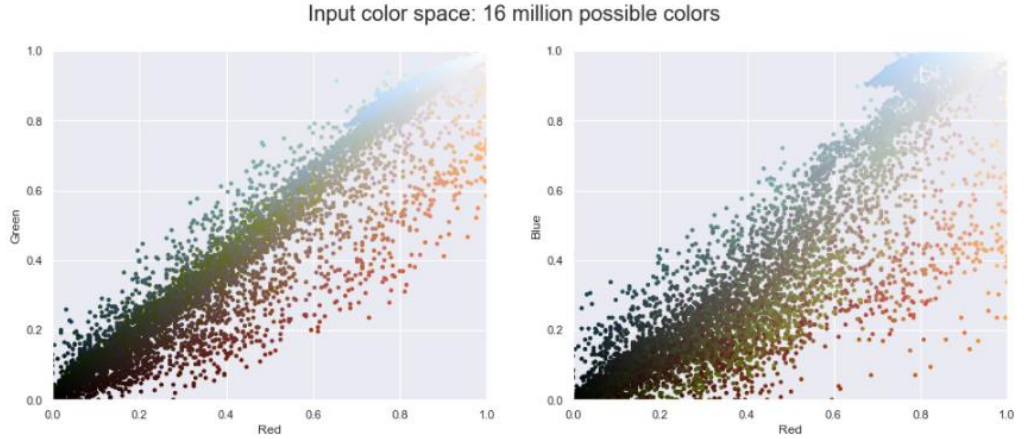
# *k*-means para la compresión de imágenes
# Visualización de los colores (2 a 2)

Sólo se viusalizan 10000 pixels por eficiencia

```
def plot_pixels(data, title, colors=None, N=10000):
    if colors is None:
        colors = data
    # choose a random subset
    rng = np.random.RandomState(0)
    i = rng.permutation(data.shape[0])[:N]
    colors = colors[i]
    R, G, B = data[i].T
    fig, ax = plt.subplots(1, 2, figsize=(16, 6))
    ax[0].scatter(R, G, color=colors, marker='.')
    ax[0].set(xlabel='Red', ylabel='Green', xlim=(0, 1), ylim=(0, 1))
    ax[1].scatter(R, B, color=colors, marker='.')
    ax[1].set(xlabel='Red', ylabel='Blue', xlim=(0, 1), ylim=(0, 1))
    fig.suptitle(title, size=20);

plot_pixels(data, title='Input color space: 16 million possible colors')
```

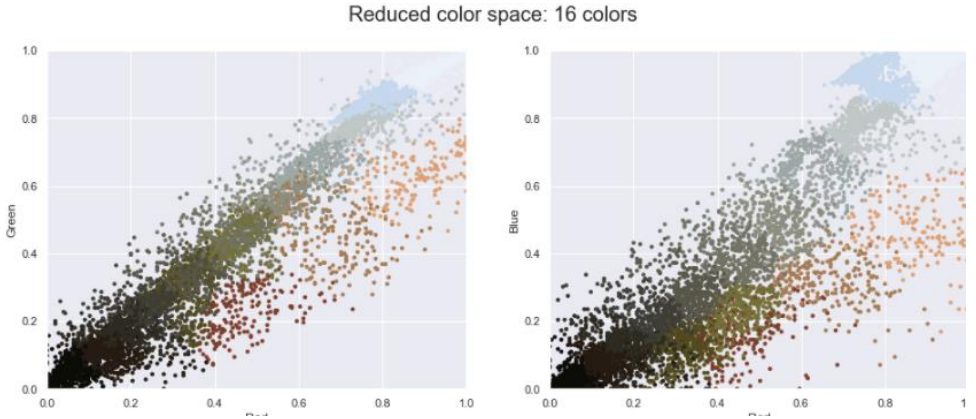
```
plot_pixels(data, title='Input color space: 16 million possible colors')
```



# *k*-means para la compresión de imágenes
# Reducción de 16M de colores a 16 colores

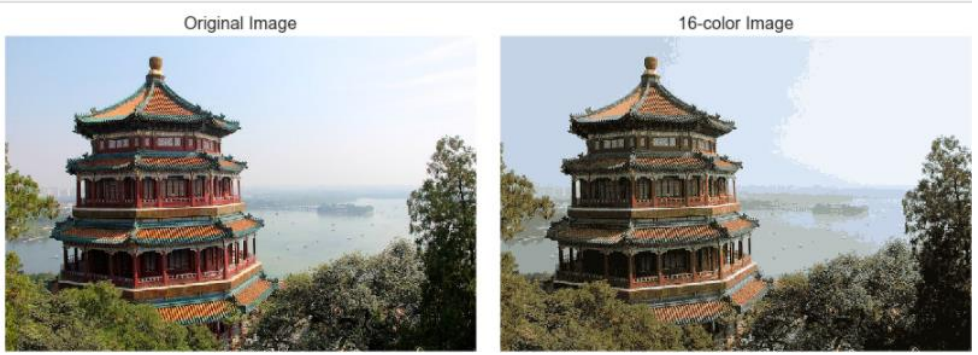
Se usa **mini batch *k*-means**, que solo usa una parte de los datos

```
import warnings; warnings.simplefilter('ignore')  # Fix NumPy issues.
from sklearn.cluster import MiniBatchKMeans
kmeans = MiniBatchKMeans(16)
kmeans.fit(data)
new_colors = kmeans.cluster_centers_[kmeans.predict(data)]
plot_pixels(data, colors=new_colors,
        title="Reduced color space: 16 colors")
```



# *k*-means para la compresión de imágenes
## Recoloreado

```
china_recolored = new_colors.reshape(china.shape)
fig, ax = plt.subplots(1, 2, figsize=(16, 6),
                       subplot_kw=dict(xticks=[], yticks=[]))
fig.subplots_adjust(wspace=0.05)
ax[0].imshow(china)
ax[0].set_title('Original Image', size=16)
ax[1].imshow(china_recolored)
ax[1].set_title('16-color Image', size=16);
```



# *k*-means para la compresión de imágenes
Imagen comprimida con factor 1 Millón



# Referencias


https://ipython.org/

https://numpy.org/

https://jupyter.org/try

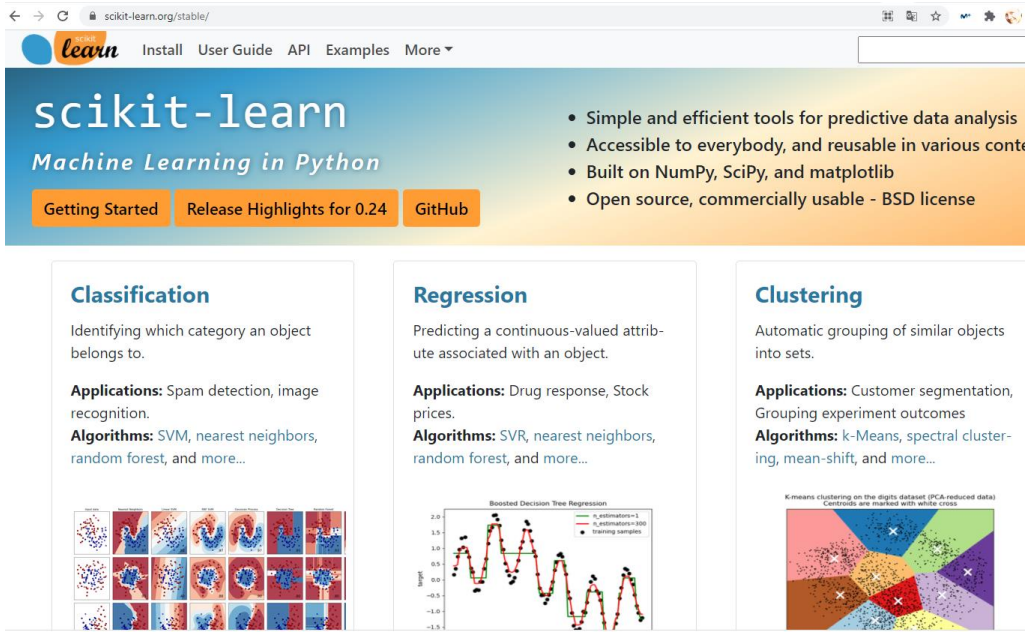
https://pandas.pydata.org/

http://seaborn.pydata.org/

https://scikit-learn.org/stable/

# Referencias

https://scikit-learn.org/stable/



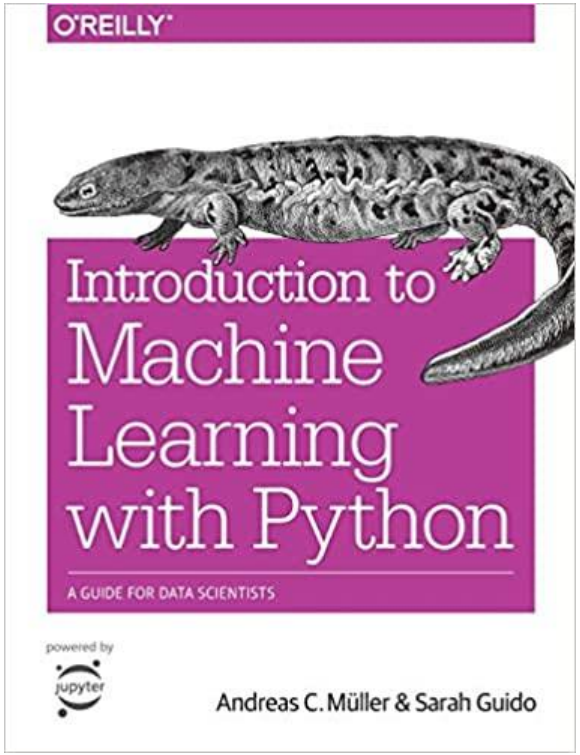
# Referencias

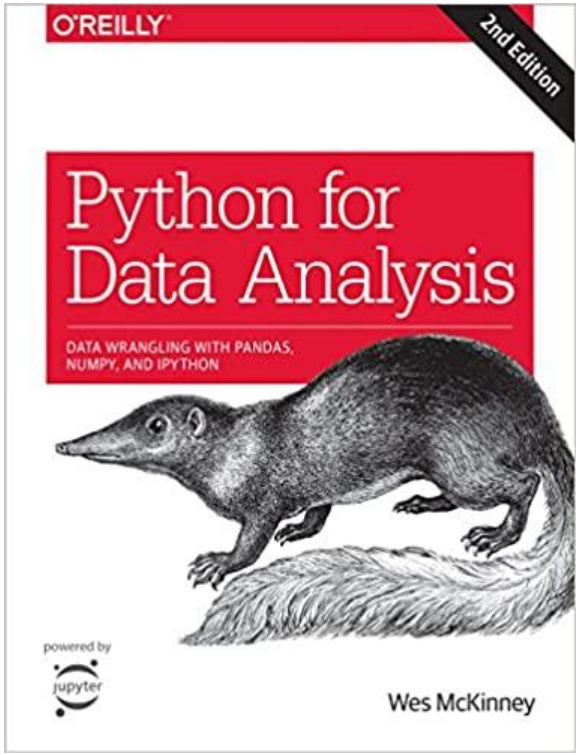
Face recognition example:

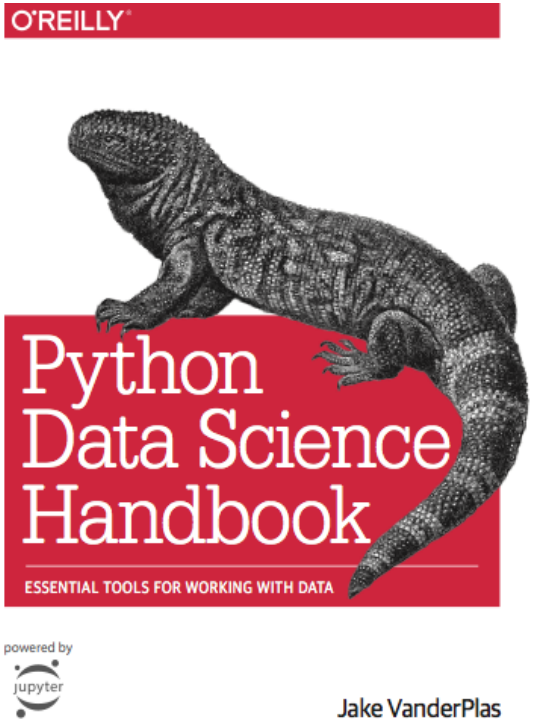
https://scikit-learn.org/stable/auto_examples/applications/plot_face_recognition.html#sphx-glr-auto-examples-applications-plot-face-recognition-py

https://jefflirion.github.io/udacity/Intro_to_Machine_Learning/Lesson12.html

# Referencias




https://jakevdp.github.io/PythonDataScienceHandbook/

ntic master
revolucionamos la comunicación

UNIVERSIDAD COMPLUTENSE DE MADRID

